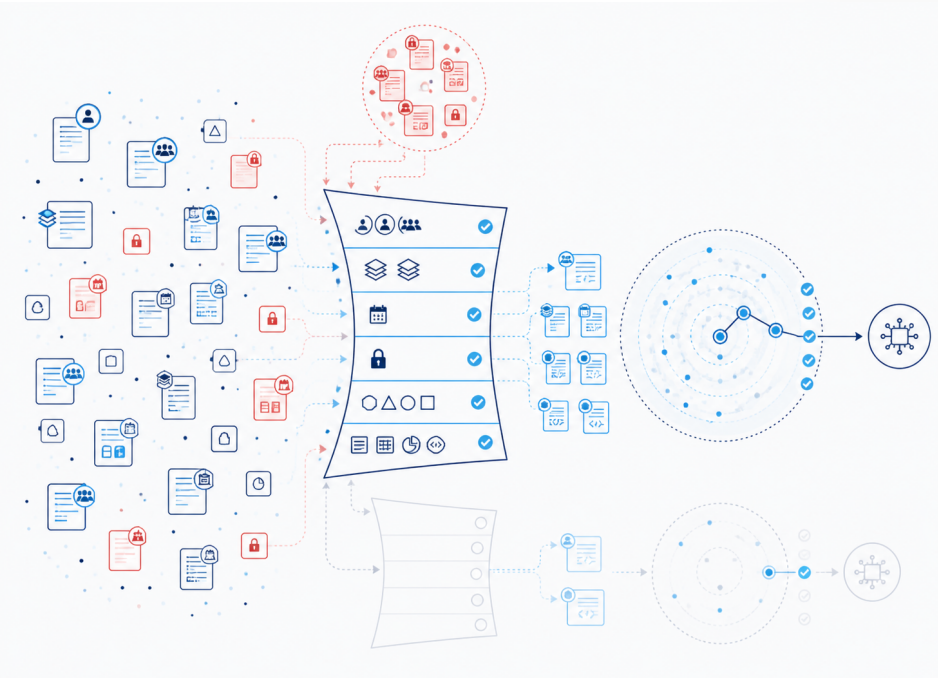
RAG RETRIEVAL

Metadata Filtering for RAG



Semantic search finds the right topic. Only metadata can tell the current doc from last year's, your tenant from another, a **policy from a rumor.**

AI Coach John FOLLOW | PROITBRIDGE Strive For Better Future

REPOST

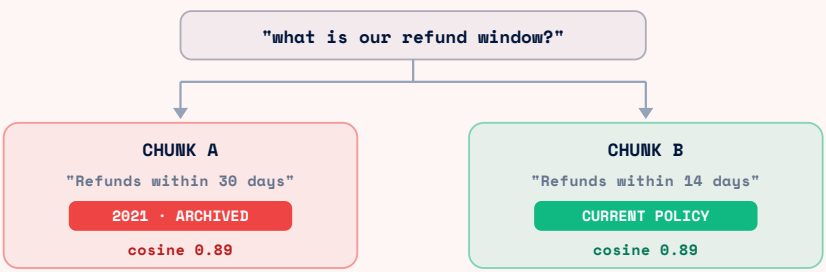
PROITBRIDGE

Right Topic, Wrong Document

PROBLEM

Embeddings rank by meaning. Two chunks that mean the same thing look identical to cosine similarity, even when one is wrong to return.

SAME MEANING, DIFFERENT TRUTH



the ranker cannot see the date — both are equally "relevant"

The failure is **silent**: a confident, well-worded answer built on the wrong source. No error is raised.

PROITBRIDGE

Metadata Is Structured Facts On Each Chunk CONCEPT

Alongside the vector, every chunk carries a small dictionary of exact-match fields you can filter on before similarity ever runs.

ONE CHUNK = TEXT + VECTOR + METADATA

```
{
  "text": "Rotate keys from Settings > API...",
  "vector": [0.021, -0.44, ...],       # semantic
  "metadata": {                        # filterable
    "product":    "api",
    "version":    3,
    "doc_type":   "guide",
    "updated_at": 20260115,
    "access":     "public"
  }
}
```

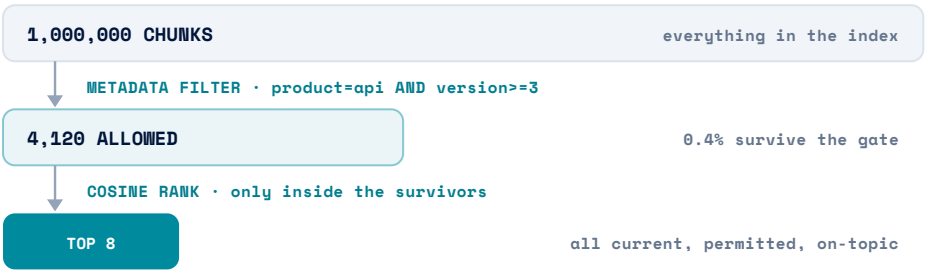
Good fields are **low-cardinality and categorical**: product, type, language, tenant, date, access level.

PROITBRIDGE

Filter First, Then Rank What Survives

HOW

A metadata filter is a hard gate. It removes candidates entirely, then similarity orders only the ones that passed.



WITHOUT FILTER

Search all 1M. The nearest neighbours include archived, other-tenant and wrong-product chunks that happen to read similarly.

WITH FILTER

Constrain to ~4k allowed chunks first. Every neighbour is current, permitted and on-topic before ranking even begins.

Filtering is not about relevance. It is about **correctness, freshness and permission.**

PROITBRIDGE

Add Metadata When You Chunk, Not Later

INGEST

Attach fields at write time. Backfilling metadata onto a live index means re-embedding or re-upserting every record.

CHROMA: ATTACH A METADATA DICT PER CHUNK

```
for doc in documents:
    for chunk in split(doc.text):
        collection.add(
            documents=[chunk],
            ids=[uid()],
            metadatas=[{                    # flat, typed values
                "product":    doc.product,
                "version":    int(doc.version),    # number, not "3"
                "doc_type":   doc.kind,
                "updated_at": doc.date_int,      # 20260115
                "access":     doc.acl,
            }],
        )
```

⚠ Store dates and versions as **numbers**, not strings. `"9" > "10"` is true for strings.

Four Filter Shapes Cover Almost Everything OPERATORS

Exact match, ranges, membership and boolean combinations. Every vector DB exposes the same core set under slightly different names.

SHAPE	OPERATOR	EXAMPLE
Exact	`$eq`	product = "api"
Range	`$gte $lte`	version >= 3, updated_at >= 20260101
Membership	`$in`	doc_type in (guide, reference)
Negation	`$ne $nin`	access != "internal"
Combine	`$and $or`	product=api AND version>=3

Ranges and `$in` are what make metadata powerful: "current versions only", "these three products".

PROITBRIDGE

Pre-Filter, Not Post-Filter

THE CHOICE

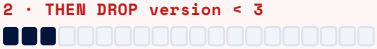
Filter before the nearest-neighbour search, not after. Post-filtering silently shrinks your results and can return too few.

✕ POST-FILTER (NAIVE)

1 · RANK ALL, TAKE TOP 20



2 · THEN DROP version < 3



3 · WHAT REACHES THE LLM



17 of 20 thrown away

You asked for 20, ranking ignored the filter, then most got thrown away. **Recall collapses.**

✓ PRE-FILTER (CORRECT)

1 · RESTRICT TO version >= 3



2 · RANK INSIDE THAT SET



3 · WHAT REACHES THE LLM



a full 20, all valid

The index searches only allowed vectors, so you always get k **real** results.

Qdrant, Pinecone and Weaviate do **filtered ANN** natively. Pass the filter into the query, do not filter afterward.

Query With The Filter Attached

RETRIEVE

The same call carries the question vector and the constraint. The database intersects them in one pass.

QDRANT: FILTERED SIMILARITY SEARCH

```
hits = client.query_points(
    collection_name="docs",
    query=embed("how do I rotate API keys?"),
    query_filter=Filter(must=[
        FieldCondition(key="product", match=Match("api")),
        FieldCondition(key="version", range=Range(gte=3)),
        FieldCondition(key="access",  match=Match("public")),
    ]),
    limit=8,
)   # -> 8 chunks, all api, all v3+, all public
```

> Index the payload fields you filter on. An unindexed filter forces a **slow full scan.**

A Support Bot, One Query End To End

EXAMPLE

A user on the Pro plan asks about v3. Watch the filter turn a noisy corpus into three trustworthy chunks.

USER ASKS	"Why is my API key rejected after upgrade?"
CONTEXT	plan = **pro**, product = **api**, running **v3**
FILTER	`product=api AND version>=3 AND access in (public, pro)`



rank happens only over the 412 that passed

No v2 docs, no internal runbooks, no other product. The LLM sees **only what is true for this user.**

PROITBRIDGE

When It Earns Its Keep And When It Hurts

JUDGEMENT

Filters are free correctness when the field is categorical. They become a liability when they are high-cardinality or over-applied.

ADD A FILTER WHEN

1. **Multi-tenant.** Never leak tenant A's data into tenant B's answers.
2. **Versioned or dated.** Only the current release or recent records are valid.
3. **Access control.** Role decides which chunks are even visible.
4. **Known facets.** Product, region, language, document type.

DO NOT FILTER WHEN

1. **Cardinality is huge.** A unique id per chunk is not a facet.
2. **You over-constrain.** Three AND clauses can empty the result set.
3. **The field is fuzzy.** "Topic" belongs in the embedding, not a filter.
4. **It is guessable.** If similarity already handles it, skip it.

Filter on **hard constraints.** Let the embedding handle **soft relevance.**

Let The LLM Write The Filter ADVANCED

Users do not type structured filters. A small extraction step turns natural language into a metadata query, called self-querying.

USER: "SHOW ME REFUND RULES UPDATED THIS YEAR FOR EU"

```
# LLM extracts structure against your known schema
{
  "query":  "refund rules",          # embed & rank
  "filter": {
    "region":     "eu",
    "doc_type":   "policy",
    "updated_at": {"$gte": 20260101}
  }
}
```

⚠ Give the extractor the **exact field names and allowed values**, or it will invent filters that match nothing.

PROITBRIDGE

Five Ways Filtering Quietly Breaks

ANTI-PATTERNS

Every one of these returns results, so nothing looks wrong. The answer is just built on the wrong or empty set.

1. **String numbers.** version stored as "10" sorts below "9". Ranges lie. Store ints.
2. **Over-filtering to empty.** Four AND clauses match zero chunks; the bot answers from nothing or hallucinates.
3. **Stale metadata.** A doc was archived but its chunk still says access=public. The filter trusts the tag.
4. **Post-filtering.** Filtering after top-k shrinks recall and returns too few results.
5. **Unindexed fields.** Filtering on a field with no payload index forces a full scan and kills latency.

Log **how many chunks passed the filter.** Zero or one is your early warning that a filter is wrong.

Symptom To Cause DEBUG

What you see in production, and the metadata mistake behind it.

SYMPTOM	LIKELY CAUSE
Answers cite old policy	no date filter, or dates stored as strings
"I don't know" too often	over-constrained AND; result set is empty
Other tenant's data leaks	tenant filter missing on the query path
Range filter misses rows	version / date stored as text, not number
Queries got slow	filtered field has no payload index
Filter matches nothing	LLM invented a value not in the schema

Most "bad retrieval" bugs are **metadata bugs**, not embedding bugs.

PROITBRIDGE

Metadata Filtering On One Screen

CHEATSHEET

Attach at ingest, pre-filter at query, keep fields categorical and typed.

IDEA	THE ONE LINE THAT MATTERS
Why	similarity finds the topic; metadata finds the right doc
What	categorical, typed fields stored on every chunk
Add it	at ingestion, as numbers for dates and versions
Pre-filter	constrain before ANN, never after top-k
Operators	eq, range (gte/lte), in, and/or
Use for	tenant, version, date, access, product, language
Avoid on	high-cardinality or fuzzy "topic" fields
Self-query	let an LLM emit the filter from the known schema
Watch	string numbers, empty sets, stale tags, missing index

Filter for **correctness**, rank for **relevance.** Two knobs, and most teams only turn one.